Proyecto Integrador

Instituto: IFTS N° 18

Carrera: Tecnicatura Superior en Ciencia de Datos e Inteligencia Artificial

Materia: Seminario de Actualización

Profesora:

Alumna: Florencia Pavon — DNI: 31.348.780

Fecha de entrega: 16/10/2026

A. El problema 1

El problema que se busca resolver es analizar los sentimientos de las personas al expresar sus opiniones sobre un producto o servicio.

Se utiliza un modelo de IA ya que el análisis de lenguaje por medio de algorifmos con reglas escritas a manos puede llevar mucho tiempo, ser costoso, pesado y no ser abarcativo a todas las variantes del idioma en español, o incluso, dentro de un mismo idioma; como por ejemplo: las jergas, los dichos, las ironías

B. El modelo 2

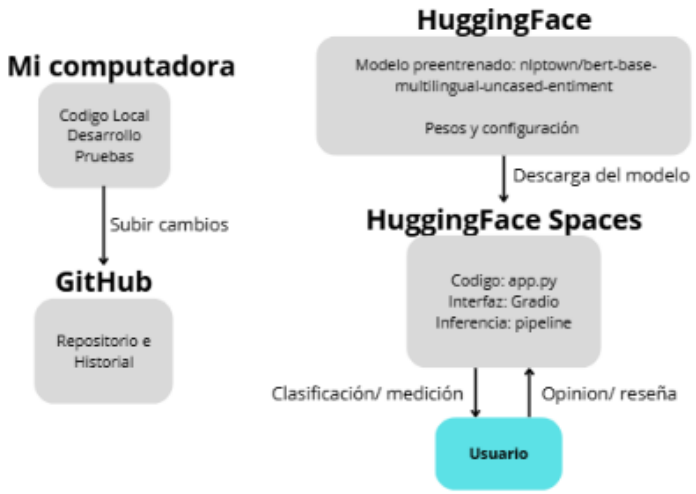
Consideré dos modelos
Modelo 1: nlptown/bert-base-multilingual-uncased-sentiment
Modelo 2: tabularisai/multilingual-sentiment-analysis

Ambos fueron entrenados con varios idiomas, me interesaba que también estén entrenados en español.

El modelo nlptown/bert-base-multilingual-uncased-sentiment se entrenó con 50k de opiniones de productos en español y Clasifica las opiniones en cinco niveles, representados por una escala de una a cinco estrellas.Mientras que tabularisai/multilingual-sentiment-analysis aprovecha datos sintéticos de múltiples fuentes y las clasifica en cinco categorías: muy negativo, negativo, neutral, positivo y muy positivo.

Ambos modelos se equivocan en una sola frase “No se, era para un regalo”, no se encuentra razón en el modo de entrenamiento, ni la cantidad de datos con el que fue entrenado. Probablemente la palabra “No” dispare una respuesta negativa, mientras que tendria que ser neutral (ver imagen 1)

C. Dónde corre cada cosa 3



	modelo	frase	label	score	tiempo_segundos	correcto
0	Modelo 1	Me encanto el producto, es excelente	5 stars	0.841069	0.372587	Sí
1	Modelo 1	No me gusto el producto, la calidad es muy mala	1 star	0.710590	0.141700	Sí
2	Modelo 1	Es genial, recomendados	5 stars	0.785776	0.197856	Sí
3	Modelo 1	Es horrible, nunca volveria a comprarlo	1 star	0.961122	0.133925	Sí
4	Modelo 1	.	3 stars	0.253934	0.036763	Sí
5	Modelo 1	Cumple con las 3 B, bueno, bonito, barato	4 stars	0.482929	0.089419	Sí
6	Modelo 1	No se, era para un regalo	1 star	0.441015	0.102468	No
7	Modelo 1	Lo barato sale caro	2 stars	0.370588	0.106393	Sí
8	Modelo 1	Ni ACME se atrevio a tanto	2 stars	0.363253	0.120014	Sí
9	Modelo 2	Me encanto el producto, es excelente	Positive	0.715037	0.847719	Sí
10	Modelo 2	No me gusto el producto, la calidad es muy mala	Negative	0.806509	0.071335	Sí
11	Modelo 2	Es genial, recomendados	Positive	0.826218	0.055895	Sí
12	Modelo 2	Es horrible, nunca volveria a comprarlo	Very Negative	0.796961	0.065824	Sí
13	Modelo 2	.	Neutral	0.588197	0.020241	Sí
14	Modelo 2	Cumple con las 3 B, bueno, bonito, barato	Positive	0.934355	0.048208	Sí
15	Modelo 2	No se, era para un regalo	Negative	0.464132	0.053664	No
16	Modelo 2	Lo barato sale caro	Negative	0.763474	0.039366	Sí
17	Modelo 2	Ni ACME se atrevio a tanto	Neutral	0.606277	0.050910	No

Imagen 1 Tabla comparativa entre modelos

D. Tu app 4

Este modelo esta infiriendo. Carga los pesos de un modelo previamente entrenado, Como el modelo ya está entrenado, la aplicación no necesita volver a entrenarlo cada vez que recibe una opinión.

En los modelos que requieren entrenamiento, el programador debe alimentar con datos para calibrar y entrenar al modelo en sí mismo, cosa que no esta sucediendo en este caso.

La instruccion “pipe1 = pipeline("text-classification", model="nlptown/bert-base-multilingual-uncased-sentiment")” carga los pesos del modelo solamente (ver imagen 2).



Imagen 2 Pipeline y pesos del modelo

Con pipeline, el modelo se ejecuta directamente en el entorno de la aplicación. En cambio, con InferenceClient, el texto se envía a un servicio de inferencia de Hugging Face, que procesa la solicitud y devuelve una respuesta.

E. ¿Dónde vive el código y dónde vive el modelo? 5

- Mi computadora: es donde desarrollo y pruebo el código localmente.
- GitHub: guarda el código y su historial si utilizo un repositorio allí.
- Hugging Face Hub: aloja el modelo preentrenado y sus pesos.
- Hugging Face Spaces: aloja y ejecuta la aplicación publicada. Al iniciar, prepara el modelo para utilizarlo.
- Inferencia: ocurre en el entorno de ejecución de Hugging Face Spaces, mediante transformers.pipeline. El usuario envía una opinión y recibe una etiqueta de estrellas junto con una puntuación de confianza.

F. ¿Usaste pipeline o InferenceClient en la app publicada? 6

Use Pipeline, el modelo 1 resulto ser el mas idoneo. En las comparaciones realizadas, el Modelo 1 presentó mayor rapidez con ambos métodos de inferencia. En cuanto al tiempo de ejecución, el Modelo 1 fue más rápido con InferenceClient, mientras que el Modelo 2 presentó un tiempo promedio ligeramente menor con pipeline. Respecto de la precisión, el Modelo 1 obtuvo mejores resultados con pipeline, mientras que ambos modelos alcanzaron el mismo valor con InferenceClient.

Por estos motivos, inicialmente elegí el Modelo 1 para el despliegue de la aplicación. Sin embargo, la implementación final se realizó en Hugging Face Spaces mediante pipeline, debido a las limitaciones de créditos del servicio de inferencia utilizado inicialmente.

G. ¿Dónde deployaste y por qué ahí? 7

La aplicación se desplegó en Hugging Face Spaces, utilizando Gradio para construir la interfaz web y transformers.pipeline para realizar la inferencia con el modelo preentrenado nlptown/bert-base-multilingual-uncased-sentiment.

Elegí esta plataforma porque permite publicar aplicaciones de inteligencia artificial y facilita la integración con modelos alojados en Hugging Face. Además, ofrece una alternativa accesible para realizar el despliegue sin tener que administrar manualmente un servidor.

Inicialmente, había desplegado la aplicación en Render utilizando InferenceClient para consultar el servicio de inferencia de Hugging Face. Sin embargo, al agotarse los créditos disponibles para ese servicio, adapté la implementación y trasladé la aplicación a Hugging Face Spaces, donde pude ejecutar el modelo directamente mediante pipeline. (ver imagenes de errores en el siguiente punto)

Comparación con otras alternativas

- **Hosting compartido:** lo descarté porque está orientado principalmente a alojar sitios web y aplicaciones convencionales. Para ejecutar un modelo de aprendizaje automático con sus dependencias de Python, PyTorch y Transformers, necesitaba un entorno que permitiera mayor control sobre la ejecución.
- **VPS (servidor privado virtual):** habría permitido instalar las dependencias y configurar el entorno de ejecución con mayor libertad. Sin embargo, habría requerido administrar el sistema operativo, las librerías, la

seguridad y el servidor, tareas que excedían las necesidades de este proyecto.
- **Nubes elásticas:** permiten ampliar o reducir los recursos informáticos según la demanda, lo cual resulta útil para aplicaciones con cargas variables. No las elegí porque mi proyecto es una aplicación pequeña y estuduantil, por lo que hubiese sido de una complejidad innecesaria.

La aplicación necesita un entorno de ejecución disponible para recibir las opiniones de los usuarios, procesarlas mediante el modelo y devolver una predicción. El servidor ejecuta el código de Python, mantiene disponible la interfaz web y realiza la inferencia cuando se envía un texto.

Esto no significa que el servidor tenga que permanecer encendido de forma ininterrumpida: según la plataforma y el plan utilizado, la aplicación puede suspenderse por inactividad y volver a iniciarse cuando alguien accede a ella. Lo importante es que exista un entorno de ejecución disponible cuando se solicite una predicción.

H Lo que se rompió 8

Durante el desarrollo surgieron dificultades al utilizar InferenceClient, ya que el servicio de inferencia de Hugging Face devolvió un error 402 Payment Required al agotarse los créditos disponibles. Para resolverlo sin contratar un servicio pago, cambié la implementación y utilicé transformers.pipeline directamente en Hugging Face Spaces. (ver imagen 3)

También fue necesario ajustar la inicialización del modelo para adaptarla al entorno ZeroGPU de la plataforma. Después de realizar estos cambios, comprobé el funcionamiento de la aplicación con opiniones positivas y negativas, obteniendo predicciones coherentes con ambos ejemplos.

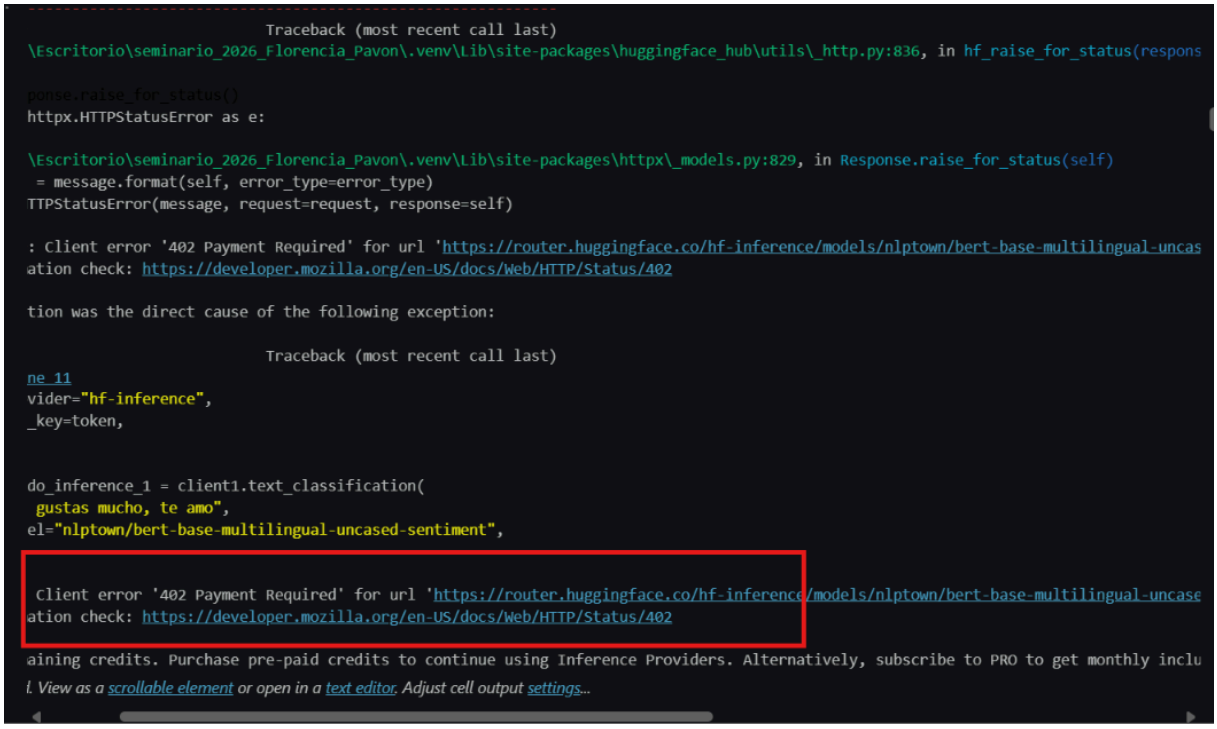


Imagen3: Error 402

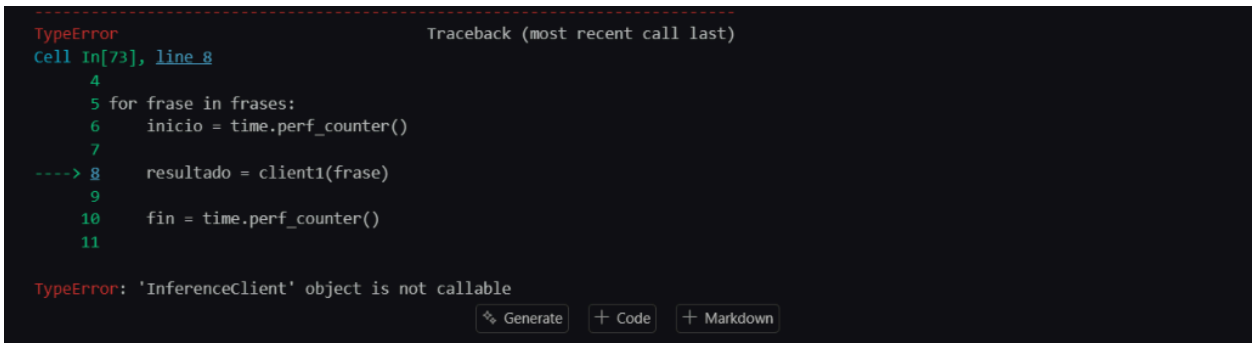


Imagen 4: Error de llamada

El error se produjo porque se intentó invocar directamente un objeto InferenceClient, en lugar de utilizar su método específico para clasificación de texto. Se corrigió utilizando text_classification()

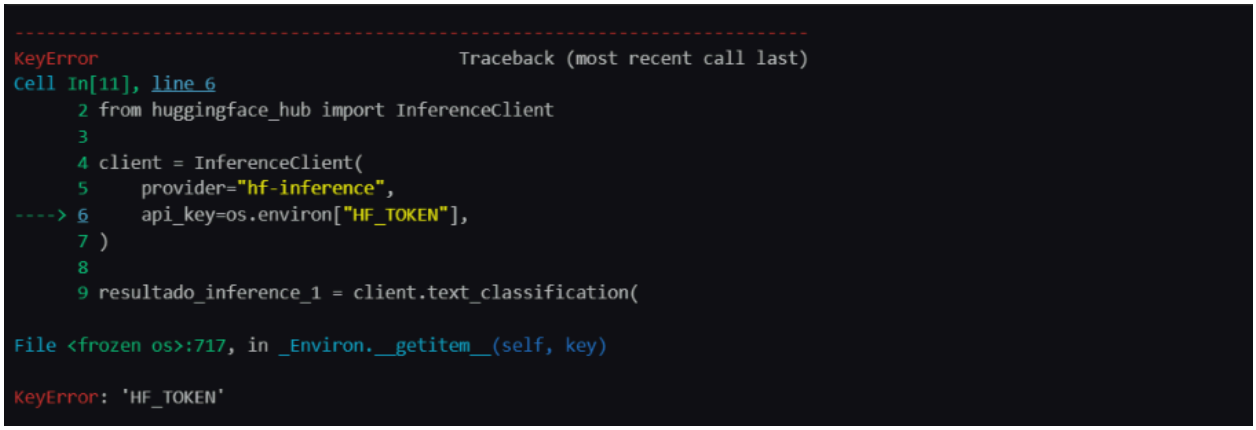


Imagen 5: Error al cargar el token

Este error se produjo porque la aplicación no lograba acceder correctamente al token de autenticación de Hugging Face mediante las variables de entorno. Para solucionarlo, se modificó el código para utilizar getpass, que permite solicitar el token por consola sin mostrarlo mientras se escribe, evitando tener que almacenarlo directamente en el código o depender de una variable de entorno previamente configurada.

I. Límites de tu solución 9

Si 1000 personas utilizaran la aplicación al mismo tiempo, podrían producirse demoras, colas de espera o errores por falta de recursos. La capacidad actual depende de los recursos disponibles en Hugging Face Spaces y de las características del plan utilizado. Para soportar esa cantidad de usuarios, sería necesario evaluar la demanda, optimizar el procesamiento y ampliar la infraestructura

Cambios en las condiciones del plan gratuito. Si Hugging Face cambiara las condiciones del plan gratuito, la aplicación podría experimentar restricciones, interrupciones o necesitar una alternativa de alojamiento. Para reducir esta dependencia, sería conveniente documentar el proceso de instalación y despliegue, mantener una copia del código y evaluar otras plataformas donde pueda ejecutarse el modelo.

Cambios necesarios para convertirla en un producto real. Sería necesario mejorar la escalabilidad, incorporar un manejo más completo de errores, monitorear el rendimiento y establecer medidas de seguridad para proteger los datos enviados por los usuarios. También sería importante realizar pruebas con una mayor variedad de opiniones para evaluar la calidad de las predicciones. Finalmente, habría que analizar los costos de infraestructura y mantenimiento, así como definir las necesidades de disponibilidad y soporte.

J Vocabulario 10

```
pipe1 = pipeline( → pipe1 es el objeto creado al ejecutar pipeline(....
    "text-classification",
    model=MODELO,
    device="cuda"
)

@spaces.GPU
def analizar_sentimiento(texto): → Parámetro de la funcion
                                        "Me encantó" sería un argumento
al llamarla.
    resultado = pipe1(texto)[0] → llamada al objeto
    etiqueta = resultado["label"] → Es una clave del diccionario que
contiene la etiqueta predicha
    puntaje = resultado["score"]

    return {
        "Etiqueta": etiqueta,
        "Confianza estimada": round(puntaje * 100, 2)
    }
```

Link a la app: https://huggingface.co/spaces/florpav/analizador-sentimientos

Link a la carpeta del TP: Repositorio

Uso de IA generativa: Uso de chatGPT para debug, ayuda con creación del deploy con Gradio en HuggingFace, ayuda para corregir sintaxis y errores de ortografía